

Staking Rewards *Billions*

HALBORN

Staking Rewards - Billions

Prepared by:  HALBORN

Last Updated 01/15/2026

Date of Engagement: January 5th, 2026 - January 5th, 2026

Summary

100% ⓘ OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ALL FINDINGS	CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
6	0	0	1	3	2

TABLE OF CONTENTS

1. Introduction	
2. Assessment summary	
3. Test approach and methodology	
4. Risk methodology	
5. Scope	
6. Assessment summary & findings overview	
7. Findings & Tech Details	
7.1 Excess reward tokens are unrecoverable when staking and reward tokens are the same	
7.2 Staking logic is incompatible with fee-on-transfer (fot) tokens	
7.3 Redundant lockduration field is stored but never used	
7.4 Expired lock state is never cleared from storage	
7.5 Single-step ownership transfer increases risk of accidental admin loss	
7.6 Locking staked tokens does not update accrued rewards	

1. Introduction

Billions engaged Halborn to conduct a security assessment of the **Billions staking rewards** contract. The assessment was performed on January 6th, 2026, with the reviewed commit hashes and in-scope contract details documented in the Scope section of this report.

The **Billions** staking implementation is based on a Synthetix-inspired reward distribution model and provides a staking framework that allows users to stake tokens and earn rewards over time. The system supports configurable reward distribution, pausing functionality, and controlled administrative operations, while introducing upgradeable architecture and time-locked staking mechanics. Upgradeability is implemented using an OpenZeppelin-compatible proxy pattern, enabling controlled logic updates while preserving on-chain state.

2. Assessment Summary

A full-time security engineer was assigned by **Halborn** to perform a targeted review of the smart contracts in scope. The engineer is a blockchain and smart contract security specialist with advanced penetration - testing and smart - contract auditing skills, and extensive knowledge of multiple blockchain protocols.

The purpose of the assessment was to:

- Identify potential security issues within the smart contracts.
- Confirm that smart contract functionality operates as intended.

In summary, **Halborn** identified several areas for improvement to minimize both the likelihood and potential impact of security risks, which were partially addressed by the **Billions team**. The primary issues included:

- **Allowed recovery of reward tokens when staking and reward tokens are the same.**
- **Restricted fee-on-transfer tokens in the staking logic.**

3. Test Approach And Methodology

Halborn performed a combination of manual code review and automated security testing to balance efficiency, timeliness, practicality, and accuracy in regard to the scope of this assessment. While manual testing is essential to uncover flaws in logic, process, and implementation, automated testing techniques enhance coverage of smart contracts and can quickly identify issues that do not follow security best practices.

The following phases and associated tools were used throughout the assessment:

- Research into the architecture, purpose, and use of the platform.
- Manual code review and walkthrough of the smart contracts to identify potential logic issues.
- Manual testing of all core functions, including deposit, withdraw, repay, and borrow, to validate expected behavior and identify edge-case vulnerabilities.
- Local testing to simulate contract interactions and validate functional and security assumptions.
- Local deployment and testing with Foundry.

4. RISK METHODOLOGY

Every vulnerability and issue observed by Halborn is ranked based on **two sets of Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker’s control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A) Specific (AO:S)	1 0.2

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Cost (AC)	Low (AC:L)	1
	Medium (AC:M)	0.67
	High (AC:H)	0.33
Attack Complexity (AX)	Low (AX:L)	1
	Medium (AX:M)	0.67
	High (AX:H)	0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

Impact Metric (M_I)	Metric Value	Numerical Value
Confidentiality (C)	None (C:N)	0
	Low (C:L)	0.25
	Medium (C:M)	0.5
	High (C:H)	0.75
	Critical (C:C)	1
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (<i>C</i>)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (<i>r</i>)	None (R:N) Partial (R:P) Full (R:F)	1 0.5 0.25
Scope (<i>s</i>)	Changed (S:C) Unchanged (S:U)	1.25 1

Severity Coefficient *C* is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score *S* is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

SEVERITY	SCORE VALUE RANGE
Critical	9 - 10
High	7 - 8.9
Medium	4.5 - 6.9
Low	2 - 4.4

SEVERITY	SCORE VALUE RANGE
Informational	0 - 1.9

5. SCOPE

REPOSITORY
(a) Repository: billions-token (b) Assessed Commit ID: 95d5f98 (c) Items in scope: contracts/staking/StakingRewards.sol Out-of-Scope: Third party dependencies and economic attacks.
REMEDIATION COMMIT ID:
https://github.com/BillionsNetwork/billions-token/pull/4/changes/c57c89206c33558af8a772d9942beac2cbbede00
Out-of-Scope: New features/implementations after the remediation commit IDs.

6. ASSESSMENT SUMMARY & FINDINGS OVERVIEW

CRITICAL 0	HIGH 0	MEDIUM 1	LOW 3	INFORMATIONAL 2
SECURITY ANALYSIS		RISK LEVEL	REMEDIATION DATE	
EXCESS REWARD TOKENS ARE UNRECOVERABLE WHEN STAKING AND REWARD TOKENS ARE THE SAME		MEDIUM	SOLVED - 01/11/2026	
STAKING LOGIC IS INCOMPATIBLE WITH FEE-ON-TRANSFER (FOT) TOKENS		LOW	RISK ACCEPTED - 01/12/2026	

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
REDUNDANT LOCKDURATION FIELD IS STORED BUT NEVER USED	LOW	RISK ACCEPTED - 01/12/2026
EXPIRED LOCK STATE IS NEVER CLEARED FROM STORAGE	LOW	RISK ACCEPTED - 01/12/2026
SINGLE-STEP OWNERSHIP TRANSFER INCREASES RISK OF ACCIDENTAL ADMIN LOSS	INFORMATIONAL	SOLVED - 01/11/2026
LOCKING STAKED TOKENS DOES NOT UPDATE ACCRUED REWARDS	INFORMATIONAL	ACKNOWLEDGED - 01/12/2026

7. FINDINGS & TECH DETAILS

7.1 EXCESS REWARD TOKENS ARE UNRECOVERABLE WHEN STAKING AND REWARD TOKENS ARE THE SAME

// MEDIUM

Description

In the `StakingRewards` contract, the design explicitly allows `stakingToken` and `rewardsToken` to be the same token. This case is handled in `notifyRewardAmount()` by subtracting `_totalSupply` from the contract balance to ensure that staked funds are not mistakenly counted as available rewards.

However, in `recoverERC20()`, only the staking token is blocked from recovery. When `rewardsToken == stakingToken`, this restriction prevents the owner from recovering any excess or unused reward tokens held by the contract, even though the accounting logic already distinguishes between staked balance and distributable rewards.

This creates an inconsistency where excess reward tokens can be safely accounted for but cannot be recovered, potentially leaving tokens permanently locked in the contract.

Code Location

In `StakingRewards` the `recoverERC20()` revert when `tokenAddress == stakingToken`.

```
371 | function recoverERC20(address tokenAddress, uint256 tokenAmount) external onlyOwner {  
372 |     require(tokenAddress != address(stakingToken), "Cannot withdraw the staking token");  
373 |     IERC20(tokenAddress).safeTransfer(owner(), tokenAmount);  
374 |     emit Recovered(tokenAddress, tokenAmount);  
375 | }
```

 Copy Code

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:M (5.0)

Recommendation

Update `recoverERC20()` to allow recovery of tokens only in excess of `_totalSupply` when `stakingToken == rewardsToken`, while continuing to prevent withdrawal of user-staked funds.

Remediation Comment

SOLVED: The **Billions team** solved the issue in the specified commit id.

Remediation Hash

<https://github.com/BillionsNetwork/billions-token/pull/4/changes/c57c89206c33558af8a772d9942beac2cbbede00>

7.2 STAKING LOGIC IS INCOMPATIBLE WITH FEE-ON-TRANSFER (FOT) TOKENS

// LOW

Description

In the `StakingRewards` contract, the `stake()` function assumes that the exact `amount` specified by the user is transferred to the contract. However, when the staking token is a fee-on-transfer (FOT) token, the actual amount received by the contract is reduced by the transfer fee.

Despite this, the contract increments `_totalSupply` and the user's `_balances` by the full `amount`, rather than the actual amount received. This leads to an accounting mismatch between the recorded staked balance and the contract's real token balance, which can later cause withdrawals or reward calculations to revert or behave unexpectedly.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:L/Y:N (2.5)

Recommendation

When staking, calculate the actual amount received by measuring the contract's token balance before and after the transfer, and update `_totalSupply` and `_balances` using the received amount rather than the user-supplied amount.

Remediation Comment

RISK ACCEPTED: The **Billions team** accepted the risk, noting that fee-on-transfer (FOT) tokens are not used in the staking system and that both staking tokens will be the BILL token, which is a standard ERC20.

7.3 REDUNDANT LOCKDURATION FIELD IS STORED BUT NEVER USED

// LOW

Description

In the `StakingRewards` contract, the `lockStake()` function assigns a value to `addressToLockedStake[msg.sender].lockDuration`. However, this field is never referenced elsewhere in the contract, and all lock enforcement logic relies solely on `unlockTimestamp`.

As a result, the `lockDuration` field is redundant and provides no functional value, increasing storage usage.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N (2.5)

Recommendation

Remove the `lockDuration` field from the `LockedStake` struct.

Remediation Comment

RISK ACCEPTED: The **Billions team** accepted the risk, noting that `LockedStake.lockDuration` is not currently used on-chain but is intended for off-chain reward calculations.

7.4 EXPIRED LOCK STATE IS NEVER CLEARED FROM STORAGE

// LOW

Description

In the `StakingRewards` contract, the `lockstate()` stored in `addressToLockedStake` is never removed after the lock has expired. While `getLockedStakeAmount()` correctly returns zero once `unlockTimestamp` has passed, the underlying `LockedStake` struct remains permanently stored.

This results in stale lock data persisting in contract storage, increasing storage usage.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (2.5)

Recommendation

Clear the user's `LockedStake` struct after `unlockTimestamp` has passed.

Remediation Comment

RISK ACCEPTED: The **Billions team** accepted the risk, noting that `LockedStake` will be used for off-chain reward calculations and is therefore intentionally retained.

7.5 SINGLE-STEP OWNERSHIP TRANSFER INCREASES RISK OF ACCIDENTAL ADMIN LOSS

// INFORMATIONAL

Description

In the `StakingRewards` contract, ownership management is implemented using OpenZeppelin's `OwnableUpgradeable`, which allows ownership to be transferred in a single transaction. This approach introduces the risk of accidental ownership transfer to an incorrect or non-functional address, potentially resulting in permanent loss of administrative control.

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:L/I:N/D:N/Y:N (1.7)

Recommendation

Replace `OwnableUpgradeable` with `Ownable2StepUpgradeable` to enforce a two-step ownership transfer process.

Remediation Comment

SOLVED: The **Billions team** solved the issue in the specified commit id.

Remediation Hash

<https://github.com/BillionsNetwork/billions-token/pull/4/changes/c57c89206c33558af8a772d9942beac2cbbede00>

7.6 LOCKING STAKED TOKENS DOES NOT UPDATE ACCRUED REWARDS

// INFORMATIONAL

Description

In the `StakingRewards` contract, the `lockStake()` function modifies the effective withdrawability of a user's staked balance by updating the lock state. However, unlike other stake-related functions such as `stake()`, `withdraw()`, and `getReward()`, `lockStake()` does not apply the `updateReward(msg.sender)` modifier.

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (1.7)

Recommendation

Add the `updateReward(msg.sender)` modifier to the `lockStake()` function to ensure rewards are consistently settled.

Remediation Comment

ACKNOWLEDGED: The **Billions team** acknowledged this issue, noting that `lockStake()` does not update the stake amount, so recalculating rewards via `updateReward(msg.sender)` is not required.

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.